Longest Substring Without Duplication

AlgoExpert · Hard · Strings

The problem

- Sliding window with last-seen index per character.

Intuition

- Slide a window; when a character repeats, jump the start past its last occurrence.

Brute force vs optimal

- Check every substring → $O(n^3)$
- Sliding window with a set → $O(n)$
- Last-seen map lets start jump directly

Approach

- 1. Map characters to last index
- 2. Move start past a repeat
- 3. Track the longest window

Worked example




```
window 'c'
best 'c'
```

'c' at 0

Worked example




```
window 'me'
best 'clem'
```

'e' at 4 repeats → move start

Worked example




```
window 'ment'
best 'clem'
```

't' at 6

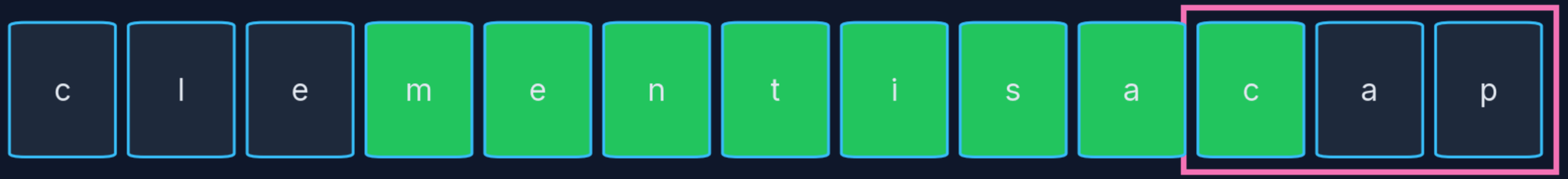
Worked example




```
window 'ca'
best 'mentisac'
```

'a' at 11 repeats → move start

Worked example




```
window 'cap'
best 'mentisac'
```

'p' at 12

Java solution

```java
public class LongestSubstringWithoutDuplication {
    public static String longestSubstringWithoutDuplication(String s) {
        Map<Character, Integer> last = new HashMap<>();
        int start = 0, bs = 0, be = 0;
        for (int i = 0; i < s.length(); i++) {
            Integer p = last.get(s.charAt(i));
            if (p != null && p >= start) start = p + 1;
            last.put(s.charAt(i), i);
            if (i + 1 - start > be - bs) { bs = start; be = i + 1; }
        }
        return s.substring(bs, be);
    }
}
```

Complexity

- Time: O(n)
- Space: O(min(n, a))

Edge cases & pitfalls

- Only jump if last ≥ start
- Empty string
- Return the substring, not length

Interview tips & follow-ups

- Mention the array[128] map

Recap

- Slide a window; when a character repeats, jump the start past its last occurrence.
- Complexity: O(n)